Assignment 1 – Heart Disease (Cleveland Database)

Group: Hatice AY, Bedia Nur ELMAS, Mahyar Naeimi

Task 1 – Exploratory Data Analysis

The analysis follows the IDA steps 1–7 from the lecture and reuses the code from the in-class fruit project where possible. The file processed.cleveland.data must be in the same folder as this notebook.

In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from matplotlib.colors import ListedColormap

pd.set_option("display.max_columns", None)   # show all columns when
printing
DATA_PATH = "processed.cleveland.data"       # relative path to the
provided dataset
```

IDA Step 1 – Big picture

Topic: coronary heart disease. Each row is one patient, each column a clinical or test result.

Source: UCI Machine Learning Repository, Cleveland Clinic Foundation (Detrano et al., 1989). The patients were referred for coronary angiography, so they are not a random sample of the population.

Question: can we predict the diagnosis num (0 = no disease, 1–4 = increasing severity) from the 13 features?

Features: age, sex, cp (chest pain type), trestbps (resting blood pressure), chol (cholesterol), fbs (fasting blood sugar > 120), restecg (resting ECG), thalach (max. heart rate), exang (exercise induced angina), oldpeak (ST depression), slope (slope of ST segment), ca (number of major vessels), thal (thallium scan result).

IDA Step 2 – Make the data machine-readable

The file has no header and missing values are written as "?". We give the column names ourselves and convert "?" to NaN.

In [2]:
```
COLUMNS = ["age", "sex", "cp", "trestbps", "chol", "fbs", "restecg",
           "thalach", "exang", "oldpeak", "slope", "ca", "thal", "num"]
df = pd.read_csv(DATA_PATH, header=None, names=COLUMNS, na_values="?")

int_cols = ["age", "sex", "cp", "trestbps", "chol", "fbs", "restecg",
            "thalach", "exang", "slope", "num"]
df[int_cols] = df[int_cols].astype(int)
df.head()
```

Out[2]:

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca
0	63	1	1	145	233	1	2	150	0	2.3	3	0.0
1	67	1	4	160	286	0	2	108	1	1.5	2	3.0
2	67	1	4	120	229	0	2	129	1	2.6	2	2.0
3	37	1	3	130	250	0	0	187	0	3.5	3	0.0
4	41	0	2	130	204	0	2	172	0	1.4	1	0.0

IDA Step 3 – Size and structure

In [3]:
```
print("shape:", df.shape)        # number of patients and columns
df.info()                        # data type and non-null count per
column
```

```
shape: (303, 14)
<class 'pandas.DataFrame'>
RangeIndex: 303 entries, 0 to 302
Data columns (total 14 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   age       303 non-null    int64
 1   sex       303 non-null    int64
 2   cp        303 non-null    int64
 3   trestbps  303 non-null    int64
 4   chol      303 non-null    int64
 5   fbs       303 non-null    int64
 6   restecg   303 non-null    int64
 7   thalach   303 non-null    int64
 8   exang     303 non-null    int64
 9   oldpeak   303 non-null    float64
 10  slope     303 non-null    int64
 11  ca        299 non-null    float64
 12  thal      301 non-null    float64
 13  num       303 non-null    int64
dtypes: float64(3), int64(11)
memory usage: 33.3 KB
```

In [4]:
```
df.describe().T.round(2)         # range, mean, quartiles per column
```

Out[4]:

	count	mean	std	min	25%	50%	75%	max
age	303.0	54.44	9.04	29.0	48.0	56.0	61.0	77.0
sex	303.0	0.68	0.47	0.0	0.0	1.0	1.0	1.0
cp	303.0	3.16	0.96	1.0	3.0	3.0	4.0	4.0
trestbps	303.0	131.69	17.60	94.0	120.0	130.0	140.0	200.0
chol	303.0	246.69	51.78	126.0	211.0	241.0	275.0	564.0
fbs	303.0	0.15	0.36	0.0	0.0	0.0	0.0	1.0
restecg	303.0	0.99	0.99	0.0	0.0	1.0	2.0	2.0
thalach	303.0	149.61	22.88	71.0	133.5	153.0	166.0	202.0
exang	303.0	0.33	0.47	0.0	0.0	0.0	1.0	1.0
oldpeak	303.0	1.04	1.16	0.0	0.0	0.8	1.6	6.2
slope	303.0	1.60	0.62	1.0	1.0	2.0	2.0	3.0
ca	299.0	0.67	0.94	0.0	0.0	0.0	1.0	3.0
thal	301.0	4.73	1.94	3.0	3.0	3.0	7.0	7.0
num	303.0	0.94	1.23	0.0	0.0	0.0	2.0	4.0

In [5]:
```
print("unique values per column:\n", df.nunique())                # few
unique values -> categorical
print("\ntarget distribution:\n", df["num"].value_counts().sort_index())
```

```
unique values per column:
 age          41
sex            2
cp             4
trestbps      50
chol         152
fbs            2
restecg        3
thalach       91
exang          2
oldpeak       40
slope          3
ca             4
thal           3
num            5
dtype: int64

target distribution:
 num
0    164
1     55
2     36
3     35
4     13
Name: count, dtype: int64
```

Observations

303 patients and 13 features plus the target. 5 features are continuous, the others are binary or categorical. For categorical columns like cp or thal the mean has no meaning. The features have very different scales (chol around 250, oldpeak around 1), so scaling is needed later. The target is imbalanced: 164 healthy and 139 diseased patients, and class 4 has only 13 patients. ca and thal have fewer than 303 values, so there are missing values.

IDA Step 4 – Missing values

In [6]:
```
print(df.isna().sum())                                    # missing
values per column
print("rows with any missing value:", df.isna().any(axis=1).sum())
df[df.isna().any(axis=1)]                                 # the
incomplete patients
```

```
age         0
sex         0
cp          0
trestbps    0
chol        0
fbs         0
restecg     0
thalach     0
exang       0
oldpeak     0
slope       0
ca          4
thal        2
num         0
dtype: int64
rows with any missing value: 6
```

Out[6]:

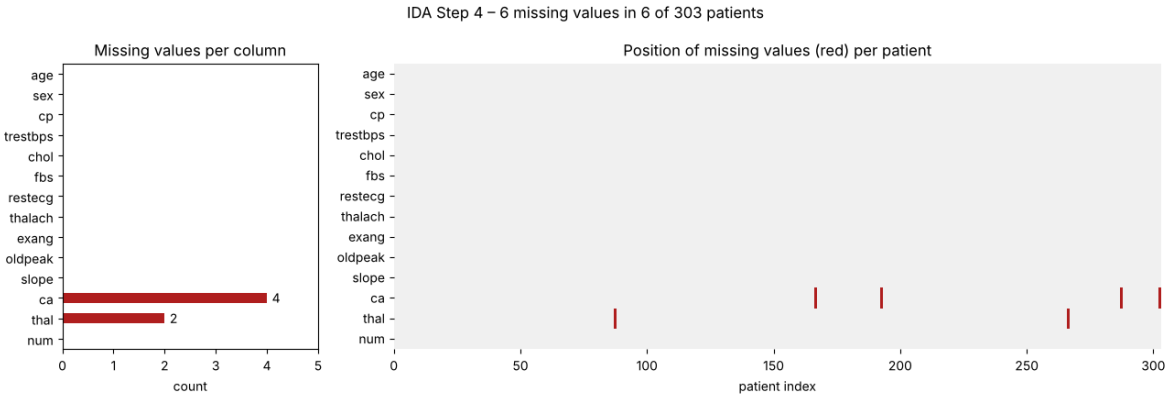
	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	
87	53	0	3	128	216	0	2	115	0	0.0	1	
166	52	1	3	138	223	0	0	169	0	0.0	1	N
192	43	1	4	132	247	1	2	143	1	0.1	2	N
266	52	1	4	128	204	1	0	156	1	1.0	2	
287	58	1	2	125	220	0	0	144	0	0.4	2	N
302	38	1	3	138	175	0	0	173	0	0.0	1	N

In [7]:
```
missing = df.isna().sum()
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(13, 4.5), gridspec_kw=
{"width_ratios": [1, 3]})

# left: number of missing values per column
missing.plot.barh(ax=ax1, color=["firebrick" if m > 0 else "lightgrey"
for m in missing])
ax1.invert_yaxis()
ax1.set(title="Missing values per column", xlabel="count", xlim=(0,
missing.max() + 1))
for i, m in enumerate(missing):
    if m > 0:
        ax1.text(m + 0.1, i, str(m), va="center")
```

```
# right: position of the missing values (red = missing)
sns.heatmap(df.isna().T, cmap=ListedColormap(["#f0f0f0", "firebrick"]),
cbar=False, ax=ax2)
ax2.set(title="Position of missing values (red) per patient",
xlabel="patient index")
ax2.set_xticks(range(0, len(df), 50))
ax2.set_xticklabels(range(0, len(df), 50), rotation=0)

plt.suptitle(f"IDA Step 4 – {missing.sum()} missing values in "
             f"{df.isna().any(axis=1).sum()} of {len(df)} patients")
plt.tight_layout()
plt.show()
```



Observations

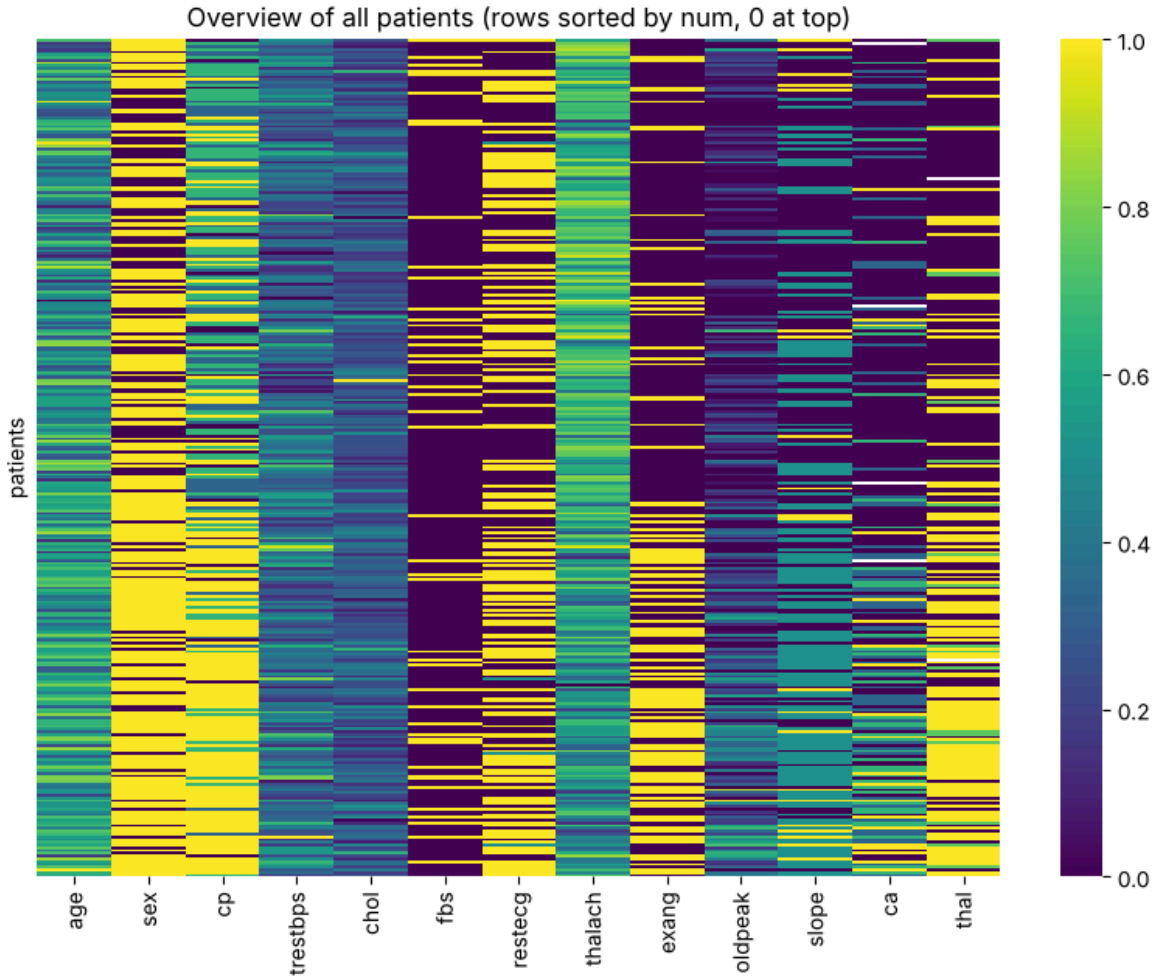
Only 6 values are missing (4 in ca, 2 in thal), each in a different patient, without a pattern. We keep all patients and fill in the missing values later with the most frequent value, inside the model pipeline to avoid data leakage.

IDA Step 5 – Overview heatmap

All patients sorted by diagnosis (healthy at the top), all features scaled to [0, 1]. A log transform, as suggested in the lecture, is not needed here because the skewness is only moderate.

In [8]:
```
features = [c for c in df.columns if c != "num"]
ordered = df.sort_values("num")                                  # sort
patients by diagnosis
scaled = (ordered[features] - ordered[features].min()) / \
         (ordered[features].max() - ordered[features].min())     # min-
max scaling to [0, 1]

fig, ax = plt.subplots(figsize=(10, 7))
sns.heatmap(scaled.values, cmap="viridis", xticklabels=features,
yticklabels=False, ax=ax)
ax.set(title="Overview of all patients (rows sorted by num, 0 at top)",
ylabel="patients")
plt.show()
```



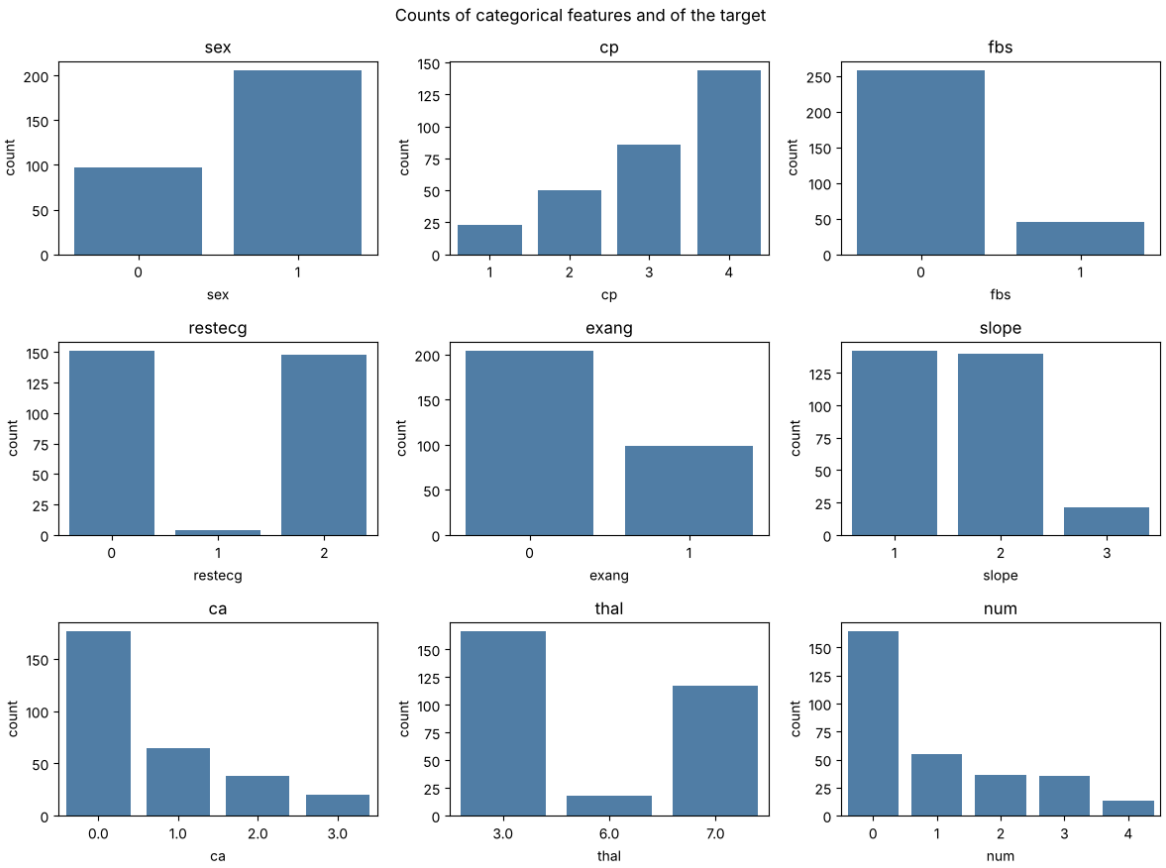
Observations

Patients with heart disease (lower part) more often have cp = 4 and exang = 1, higher oldpeak, ca and thal, and a lower thalach. chol, fbs and trestbps show no visible difference.

IDA Step 6 – Univariate analysis

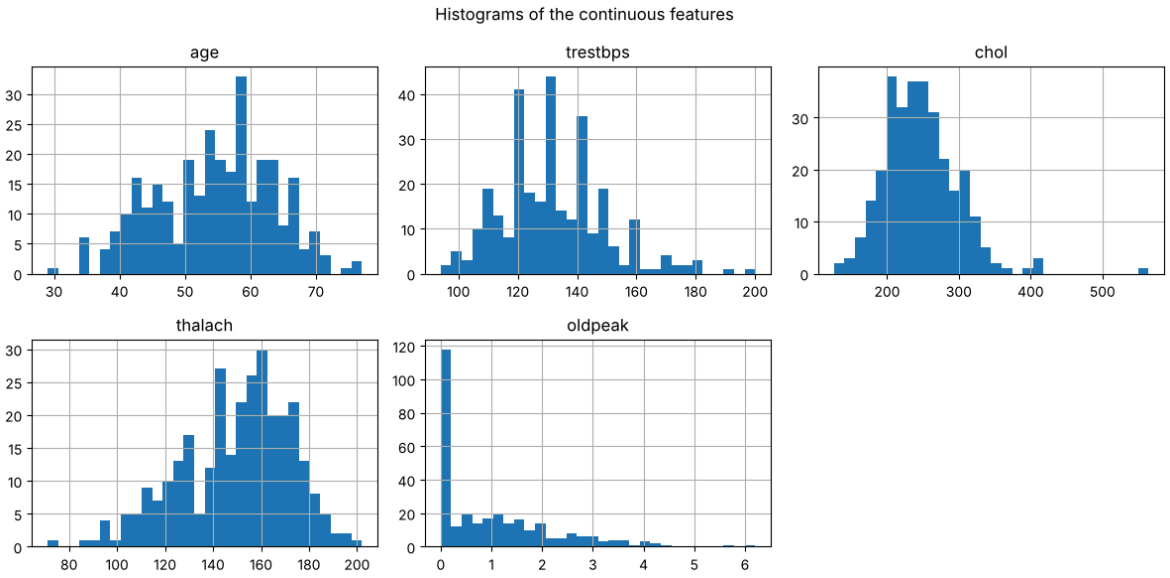
Count plots for the categorical features and the target.

In [9]:
```
cat_cols = ["sex", "cp", "fbs", "restecg", "exang", "slope", "ca",
"thal", "num"]
fig, axes = plt.subplots(3, 3, figsize=(12, 9))
for ax, col in zip(axes.ravel(), cat_cols):
    sns.countplot(x=col, data=df, ax=ax, color="steelblue")
    ax.set_title(col)
plt.suptitle("Counts of categorical features and of the target")
plt.tight_layout()
plt.show()
```



Histograms for the continuous features.

In [10]:
```
num_cols = ["age", "trestbps", "chol", "thalach", "oldpeak"]
df[num_cols].hist(bins=30, figsize=(12, 6), layout=(2, 3))
plt.suptitle("Histograms of the continuous features")
plt.tight_layout()
plt.show()
```

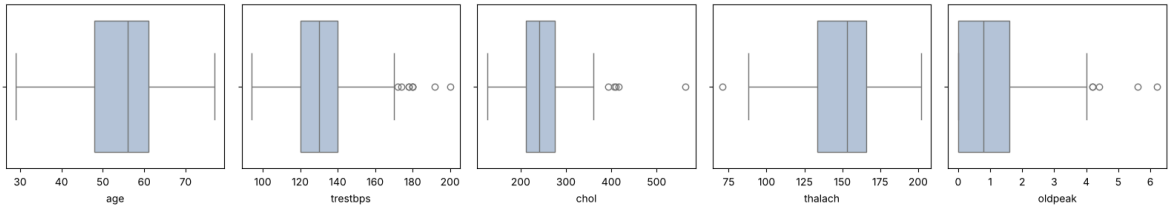


Box plots and skewness to find outliers and skewed distributions.

In [11]:
```
print("skewness:\n", df[num_cols].skew().round(2))
fig, axes = plt.subplots(1, 5, figsize=(16, 3))
for ax, col in zip(axes, num_cols):
    sns.boxplot(x=df[col], ax=ax, color="lightsteelblue")
```

```
plt.tight_layout()
plt.show()
```

```
skewness:
 age         -0.21
trestbps     0.71
chol         1.14
thalach     -0.54
oldpeak      1.27
dtype: float64
```



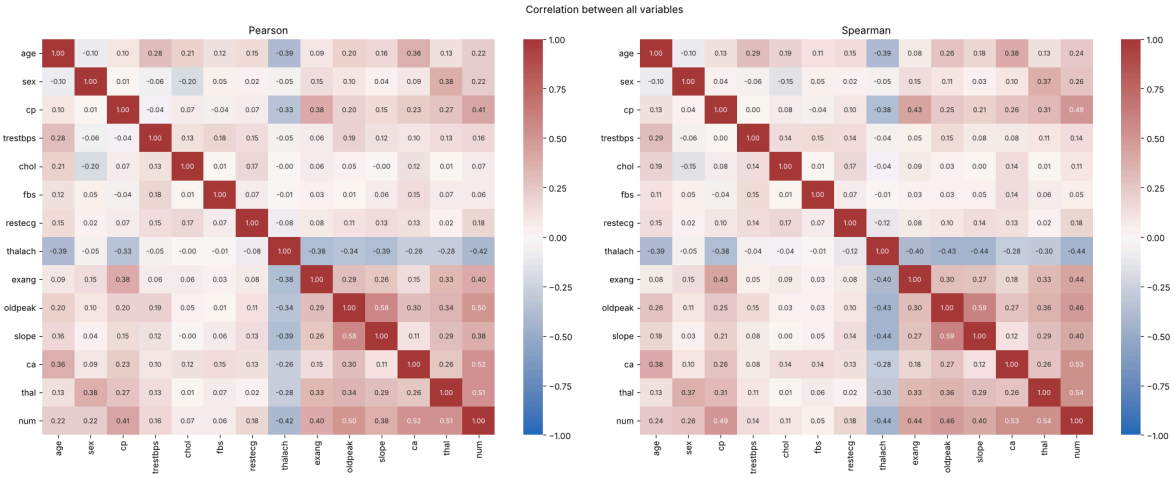
Observations

There are about twice as many men as women (206 vs. 97). Most patients have cp = 4 (asymptomatic). fbs = 1 is rare (15 %) and restecg = 1 occurs only 4 times. age is roughly symmetric, thalach slightly left-skewed, chol and oldpeak are right-skewed. chol has one extreme value (564) and trestbps some high values (up to 200). These values are medically plausible, so we keep them.

IDA Step 7 – Relationships between variables

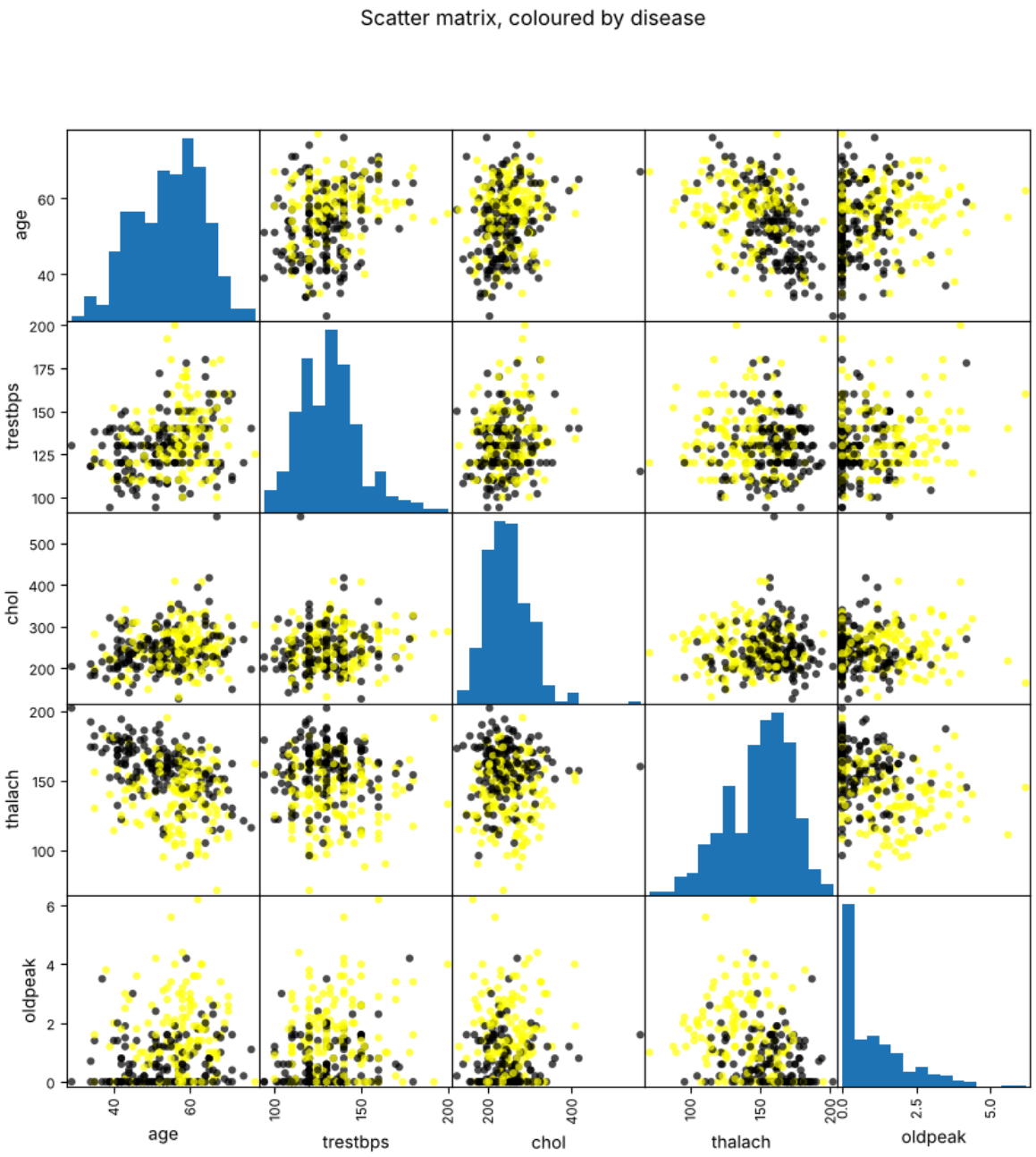
Pearson and Spearman correlation. We use Spearman for the interpretation because it is robust to outliers and suited for ordinal values.

In [12]:
```
fig, axes = plt.subplots(1, 2, figsize=(20, 8))
for ax, method in zip(axes, ["pearson", "spearman"]):
    corr = df.corr(method=method)
    sns.heatmap(corr, annot=True, fmt=".2f", cmap="vlag", vmin=-1,
vmax=1,
                ax=ax, annot_kws={"size": 8})
    ax.set_title(method.capitalize())
plt.suptitle("Correlation between all variables")
plt.tight_layout()
plt.show()
```



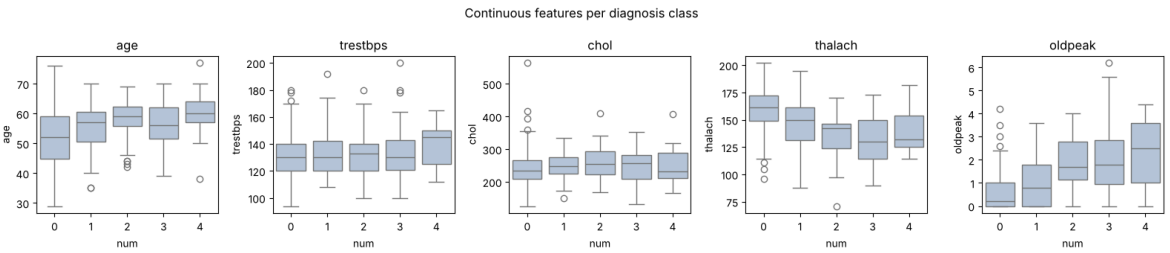
Scatter matrix of the continuous features (black = no disease, yellow = disease).

In [13]:
```
pd.plotting.scatter_matrix(df[num_cols], c=(df["num"] > 0).astype(int),
marker="o", s=20,
                           hist_kwds={"bins": 15}, figsize=(10, 10),
                           cmap=plt.get_cmap("gnuplot"), alpha=0.7)
plt.suptitle("Scatter matrix, coloured by disease")
plt.show()
```



Continuous features per diagnosis class.

In [14]:
```
fig, axes = plt.subplots(1, 5, figsize=(16, 3.5))
for ax, col in zip(axes, num_cols):
    sns.boxplot(x="num", y=col, data=df, ax=ax, color="lightsteelblue")
    ax.set_title(col)
plt.suptitle("Continuous features per diagnosis class")
plt.tight_layout()
plt.show()
```



Observations

Most features are only weakly or moderately correlated with each other. The strongest pair is oldpeak and slope (0.59), so no feature needs to be removed. The features most related to the target are thal (0.54), ca (0.53), cp (0.49), oldpeak (0.46), exang (0.44) and thalach (-0.44). fbs, chol and trestbps are almost unrelated to the diagnosis. In the scatter matrix healthy and diseased patients overlap, so we expect a good but not perfect classifier. The disease classes 1–4 overlap strongly, so predicting the severity will be much harder than predicting disease vs. no disease.

Task 2 – Classifiers for the 5-class target (num = 0 to 4)

We train four classifiers (Logistic Regression, kNN, Random Forest, SVM) and compare them with a baseline that always predicts the most frequent class. Before training, missing values are filled with the most frequent value, the nominal features cp, restecg, slope and thal are one-hot encoded, and all features are standardised. This is done inside a pipeline, so it is learned on the training data only.

In [15]:
```
from sklearn.base import clone
from sklearn.compose import ColumnTransformer
from sklearn.dummy import DummyClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix
from sklearn.model_selection import RepeatedStratifiedKFold,
StratifiedKFold, cross_val_predict
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.svm import SVC

RANDOM_STATE = 42

features = [c for c in df.columns if c != "num"]
X = df[features]
y_multi = df["num"].values

NOMINAL = ["cp", "restecg", "slope", "thal"]      # labels, not real
numbers -> one-hot
OTHER = [c for c in features if c not in NOMINAL]

def make_preprocessor():
    nominal = Pipeline([("impute",
SimpleImputer(strategy="most_frequent")),
                        ("onehot",
```

```
OneHotEncoder(handle_unknown="ignore", sparse_output=False))])
    other = Pipeline([("impute",
SimpleImputer(strategy="most_frequent"))])
    return Pipeline([("columns", ColumnTransformer([("nom", nominal,
NOMINAL),
                                                    ("oth", other,
OTHER)])),
                     ("scale", StandardScaler())])

def make_models():
    return {
        "Baseline (most frequent)":
DummyClassifier(strategy="most_frequent"),
        "Logistic Regression": LogisticRegression(max_iter=2000),
        "kNN (k=15)": KNeighborsClassifier(n_neighbors=15),
        "Random Forest": RandomForestClassifier(n_estimators=300,
random_state=RANDOM_STATE),
        "SVM (RBF)": SVC(kernel="rbf", random_state=RANDOM_STATE),
    }

def pipe(model):
    # preprocessing is fitted on the training folds only -> no data
leakage
    return Pipeline([("prep", make_preprocessor()), ("model", model)])
```

Evaluation

With only 303 patients a single train/test split is very noisy, so we use stratified 5-fold cross-validation repeated 10 times and report mean and standard deviation. Sensitivity and specificity are defined for two classes, so for the 5 classes we compute them for each class against the rest and take the average.

In [16]:
```
def sens_spec(y_true, y_pred, labels):
    # one-vs-rest sensitivity and specificity per class, then macro
average
    cm = confusion_matrix(y_true, y_pred, labels=labels)
    tp = np.diag(cm)
    fn = cm.sum(axis=1) - tp
    fp = cm.sum(axis=0) - tp
    tn = cm.sum() - tp - fn - fp
    with np.errstate(invalid="ignore", divide="ignore"):
        sens = tp / (tp + fn)
        spec = tn / (tn + fp)
    if len(labels) == 2:                        # binary case: positive
class = 1
        return sens[1], spec[1]
    return np.nanmean(sens), np.nanmean(spec)

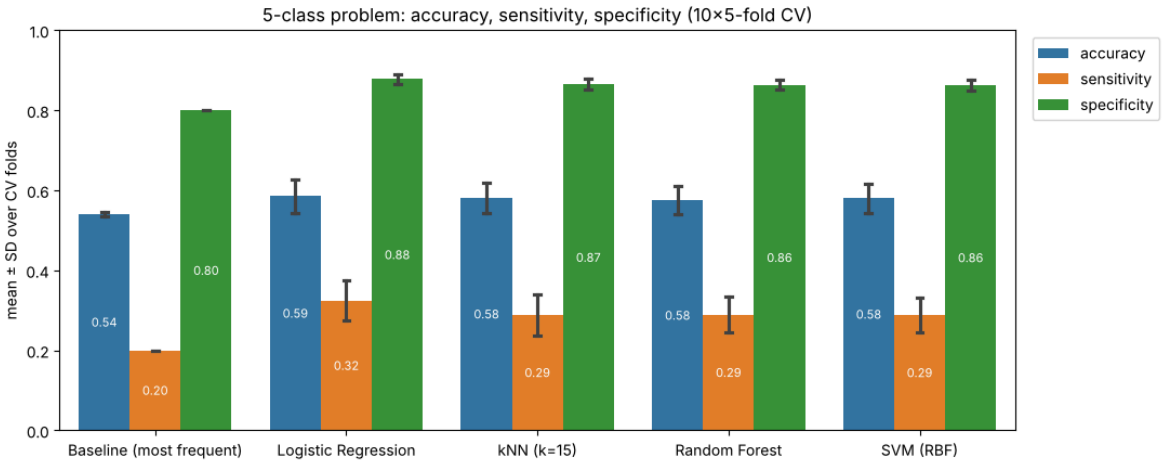
def evaluate(y, labels):
    # stratified 5-fold CV, repeated 10 times = 50 train/test splits
    cv = RepeatedStratifiedKFold(n_splits=5, n_repeats=10,
random_state=RANDOM_STATE)
    rows = []
    for name, model in make_models().items():
        for tr, te in cv.split(X, y):
            m = pipe(clone(model)).fit(X.iloc[tr], y[tr])
            pred = m.predict(X.iloc[te])
            se, sp = sens_spec(y[te], pred, labels)
```

```
            rows.append({"model": name, "accuracy":
accuracy_score(y[te], pred),
                         "sensitivity": se, "specificity": sp})
    return pd.DataFrame(rows)

def grouped_bar(res, title):
    long = res.melt(id_vars="model", var_name="metric",
value_name="value")
    fig, ax = plt.subplots(figsize=(11, 4.5))
    sns.barplot(data=long, x="model", y="value", hue="metric",
errorbar="sd", capsize=0.1, ax=ax)
    ax.set(ylim=(0, 1), ylabel="mean ± SD over CV folds", xlabel="",
title=title)
    ax.legend(loc="upper left", bbox_to_anchor=(1.01, 1))
    for c in ax.containers:
        if hasattr(c, "datavalues"):
            ax.bar_label(c, fmt="%.2f", fontsize=8, label_type="center",
color="white")
    plt.tight_layout()
    plt.show()

res_multi = evaluate(y_multi, labels=[0, 1, 2, 3, 4])
print(res_multi.groupby("model", sort=False).agg(["mean",
"std"]).round(3))
grouped_bar(res_multi, "5-class problem: accuracy, sensitivity,
specificity (10x5-fold CV)")
```

	accuracy		sensitivity		specificity
	mean	std	mean	std	mean
std					
model					
Baseline (most frequent)	0.541	0.005	0.200	0.000	0.800
0.000					
Logistic Regression	0.586	0.043	0.324	0.050	0.878
0.013					
kNN (k=15)	0.582	0.037	0.289	0.051	0.865
0.013					
Random Forest	0.575	0.035	0.290	0.046	0.864
0.012					
SVM (RBF)	0.581	0.037	0.289	0.043	0.863
0.013					

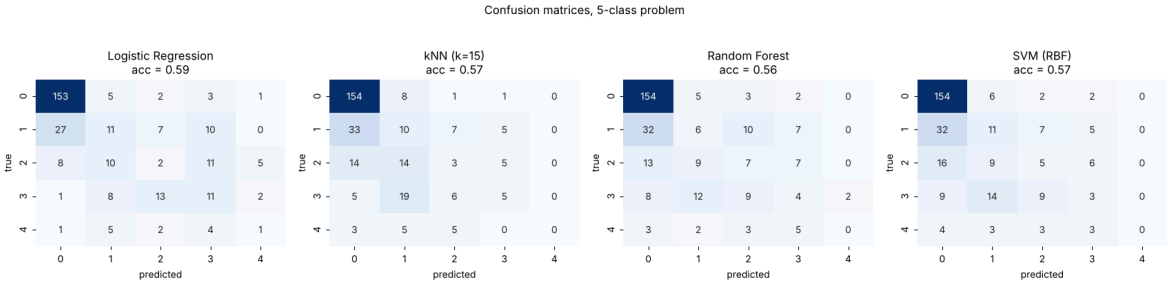


Confusion matrices

Each patient is predicted once by a model that did not see this patient during training.

In [17]:
```
def confusion_heatmaps(y, labels, title):
    cv = StratifiedKFold(n_splits=5, shuffle=True,
random_state=RANDOM_STATE)
    models = {k: v for k, v in make_models().items() if not
k.startswith("Baseline")}
    fig, axes = plt.subplots(1, len(models), figsize=(4.3 * len(models),
4))
    preds = {}
    for ax, (name, model) in zip(axes, models.items()):
        pred = cross_val_predict(pipe(model), X, y, cv=cv)
        preds[name] = pred
        cm = confusion_matrix(y, pred, labels=labels)
        sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", cbar=False,
ax=ax,
                    xticklabels=labels, yticklabels=labels)
        ax.set(title=f"{name}\nacc = {accuracy_score(y, pred):.2f}",
               xlabel="predicted", ylabel="true")
    plt.suptitle(title, y=1.04)
    plt.tight_layout()
    plt.show()
    return preds

preds_multi = confusion_heatmaps(y_multi, [0, 1, 2, 3, 4], "Confusion
matrices, 5-class problem")
```



Observations

All models reach an accuracy of only about 0.58, which is barely better than the baseline (0.54) that always predicts "no disease". The average sensitivity is low (about 0.3): the confusion matrices show that most class 0 patients are classified correctly, but patients of classes 1 to 4 are often predicted as healthy or confused with each other, and class 4 is almost never found. The specificity looks high (about 0.87), but this is partly because in a one-vs-rest view most patients are negatives for each class. The reason is that there are few patients per severity class and the features hardly differ between classes 1 to 4.

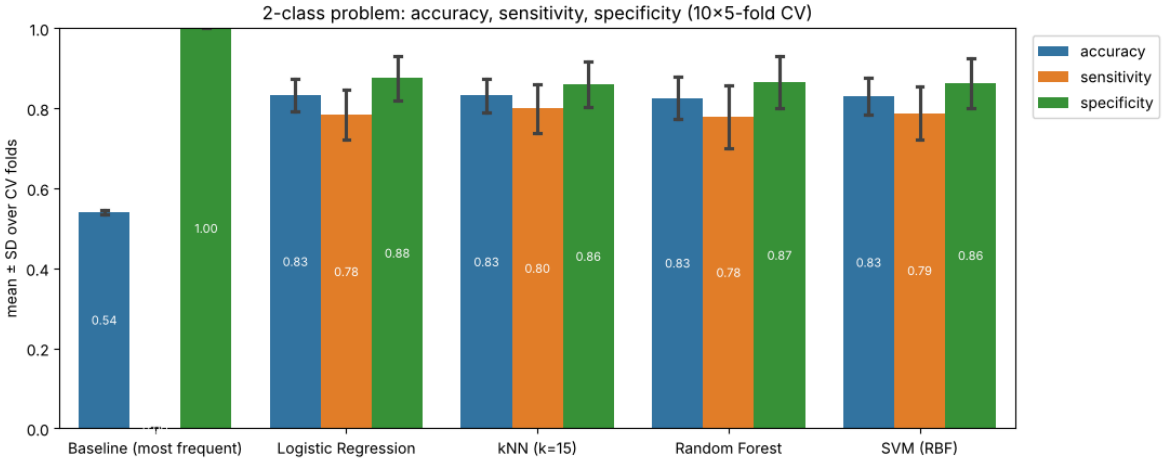
Task 3 – Heart disease vs. no heart disease

We repeat Task 2 with only two classes: 0 = no disease (num = 0) and 1 = disease (num = 1 to 4). Sensitivity is now the share of diseased patients that are detected, specificity the share of healthy patients that are classified as healthy.

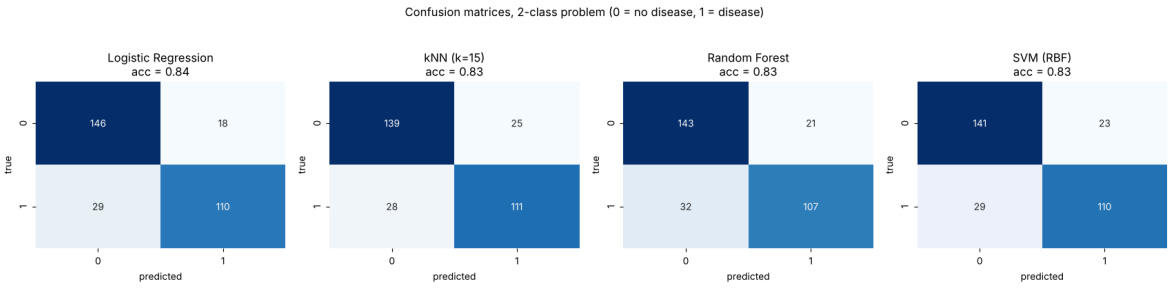
In [18]:
```
y_bin = (df["num"] > 0).astype(int).values
print(pd.Series(y_bin).value_counts().rename({0: "no disease", 1:
"disease"}))
```

```
res_bin = evaluate(y_bin, labels=[0, 1])
print(res_bin.groupby("model", sort=False).agg(["mean",
"std"]).round(3))
grouped_bar(res_bin, "2-class problem: accuracy, sensitivity,
specificity (10x5-fold CV)")
```

```
no disease    164
disease       139
Name: count, dtype: int64
                         accuracy        sensitivity        specificity
                             mean    std        mean    std        mean
std
model
Baseline (most frequent)    0.541  0.005       0.000  0.000       1.000
0.000
Logistic Regression         0.834  0.041       0.783  0.062       0.876
0.055
kNN (k=15)                  0.832  0.043       0.800  0.062       0.860
0.058
Random Forest               0.826  0.053       0.779  0.078       0.865
0.066
SVM (RBF)                   0.829  0.046       0.789  0.066       0.864
0.062
```



In [19]:
```
preds_bin = confusion_heatmaps(y_bin, [0, 1], "Confusion matrices, 2-
class problem (0 = no disease, 1 = disease)")
```



Comparison with the 5-class models

We also take the predictions of the 5-class models from Task 2, merge the classes 1 to 4 into "disease", and compare their accuracy with the binary models.

In [20]:
```
y_multi = df["num"].values
cv = StratifiedKFold(n_splits=5, shuffle=True,
random_state=RANDOM_STATE)
rows = []
```

```
for name, model in make_models().items():
    if name.startswith("Baseline"):
        continue
    pred_multi = cross_val_predict(pipe(model), X, y_multi, cv=cv)
    rows.append({"model": name,
                 "5-class model, merged to 2 classes":
accuracy_score(y_bin, (pred_multi > 0).astype(int)),
                 "binary model": accuracy_score(y_bin,
preds_bin[name])})
print(pd.DataFrame(rows).set_index("model").round(3))
```

```
                    5-class model, merged to 2 classes  binary model
model
Logistic Regression                                0.842         0.845
kNN (k=15)                                         0.785         0.825
Random Forest                                      0.782         0.825
SVM (RBF)                                          0.766         0.828
```

Analysis

With two classes the accuracy rises from about 0.58 to about 0.83, the sensitivity from about 0.3 to about 0.79, and the specificity stays around 0.87. All four models perform almost equally. The baseline now has sensitivity 0 and specificity 1 because it always predicts "no disease", which shows that accuracy alone is not enough.

The change works because both classes are now large and almost balanced (164 vs. 139), and because the features separate healthy from diseased patients well, but not the severity classes (see IDA step 7). Even the 5-class models reach a much higher accuracy when their predictions are merged to two classes, but the models trained directly on the binary question are still as good or better.

Does it make sense? Yes. Clinically, the first question is whether a patient has a significant narrowing of a coronary artery, and this is also the definition used by Detrano et al. (1989). With only 13 to 36 patients per severity class, a 5-class model cannot be learned reliably. The disadvantage is that we lose information: a patient with severe disease is treated the same as a patient with mild disease, and the order of the classes is ignored. Also, a missed disease (false negative) is worse than a false alarm (false positive), so in practice one would choose a threshold with a higher sensitivity (see Task 4).

Task 4 – ROC curves for the 2-class problem

For each classifier we compute a score for "disease" for every patient with 5-fold cross-validation (each patient is scored by a model that did not see it). For Logistic Regression, kNN and Random Forest this is the predicted probability, for the SVM it is the distance to the decision boundary. The ROC curve shows the sensitivity against 1 − specificity for all possible thresholds, and the AUC is the area under the curve.

In [21]:
```
from sklearn.metrics import roc_curve, auc

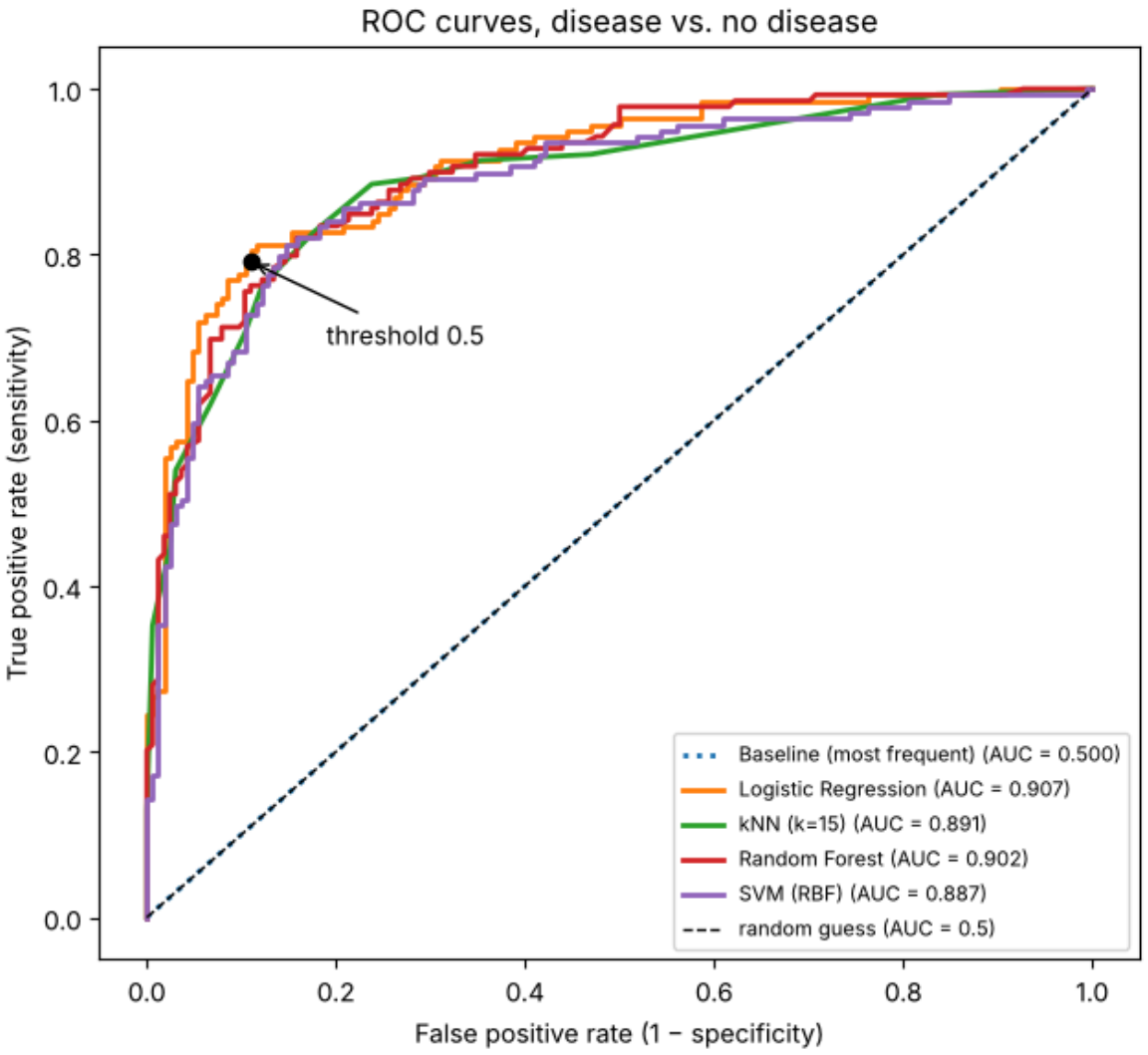
y_bin = (df["num"] > 0).astype(int).values
```

```
cv = StratifiedKFold(n_splits=5, shuffle=True,
random_state=RANDOM_STATE)

fig, ax = plt.subplots(figsize=(6.5, 6))
for name, model in make_models().items():
    if hasattr(model, "predict_proba"):
        score = cross_val_predict(pipe(model), X, y_bin, cv=cv,
method="predict_proba")[:, 1]
        default_threshold = 0.5
    else:   # SVM: distance to the decision boundary, default threshold
0
        score = cross_val_predict(pipe(model), X, y_bin, cv=cv,
method="decision_function")
        default_threshold = 0.0
    fpr, tpr, thr = roc_curve(y_bin, score)
    style = ":" if name.startswith("Baseline") else "-"
    ax.plot(fpr, tpr, style, lw=2, label=f"{name} (AUC = {auc(fpr,
tpr):.3f})")
    if name == "Logistic Regression":
        i = np.argmin(np.abs(thr - default_threshold))   # point of the
default threshold 0.5
        ax.plot(fpr[i], tpr[i], "ko")
        ax.annotate("threshold 0.5", (fpr[i], tpr[i]), xytext=(fpr[i] +
0.08, tpr[i] - 0.1),
                    arrowprops={"arrowstyle": "->"})
        print(f"Logistic Regression at threshold 0.5: sensitivity =
{tpr[i]:.2f}, "
              f"specificity = {1 - fpr[i]:.2f}")

ax.plot([0, 1], [0, 1], "k--", lw=1, label="random guess (AUC = 0.5)")
ax.set(xlabel="False positive rate (1 − specificity)", ylabel="True
positive rate (sensitivity)",
       title="ROC curves, disease vs. no disease")
ax.legend(loc="lower right", fontsize=8)
plt.tight_layout()
plt.show()
```

```
Logistic Regression at threshold 0.5: sensitivity = 0.79, specificity =
0.89
```



Interpretation

All four classifiers are far above the diagonal, which stands for random guessing (AUC = 0.5). The baseline lies exactly on the diagonal because it gives every patient the same score.

The AUC is about 0.89 to 0.91 for all models. This means: if we pick one random diseased and one random healthy patient, the model gives the diseased patient the higher score in about 90 % of the cases. The curves cross each other and the differences are very small, so no model is clearly better. The simple Logistic Regression is as good as the more complex Random Forest and SVM.

The ROC curve also shows the trade-off between sensitivity and specificity. With the default threshold 0.5 (black point) Logistic Regression finds about 79 % of the diseased patients with a specificity of about 89 %. For screening, where a missed disease is worse than a false alarm, we would lower the threshold and move to the right on the curve: for example, a sensitivity of about 0.9 is possible with a specificity of about 0.7.